Solution: Maximum Product Subarray

Problem Statement

Given an integer array, find the contiguous subarray (at least one number in it) that has the maximum product. Return this maximum product.

Examples

1. - **Input**: [2,3,-2,4]
   - **Expected Output**: 6
   - **Justification**: The subarray [2,3] has the maximum product of 6.
2. - **Input**: [-2,0,-1]
   - **Expected Output**: 0
   - **Justification**: The subarray [0] has the maximum product of 0.
3. - **Input**: [-2,3,2,-4]
   - **Expected Output**: 48
   - **Justification**: The subarray [-2,3,2,-4] has the maximum product of 48.

Solution

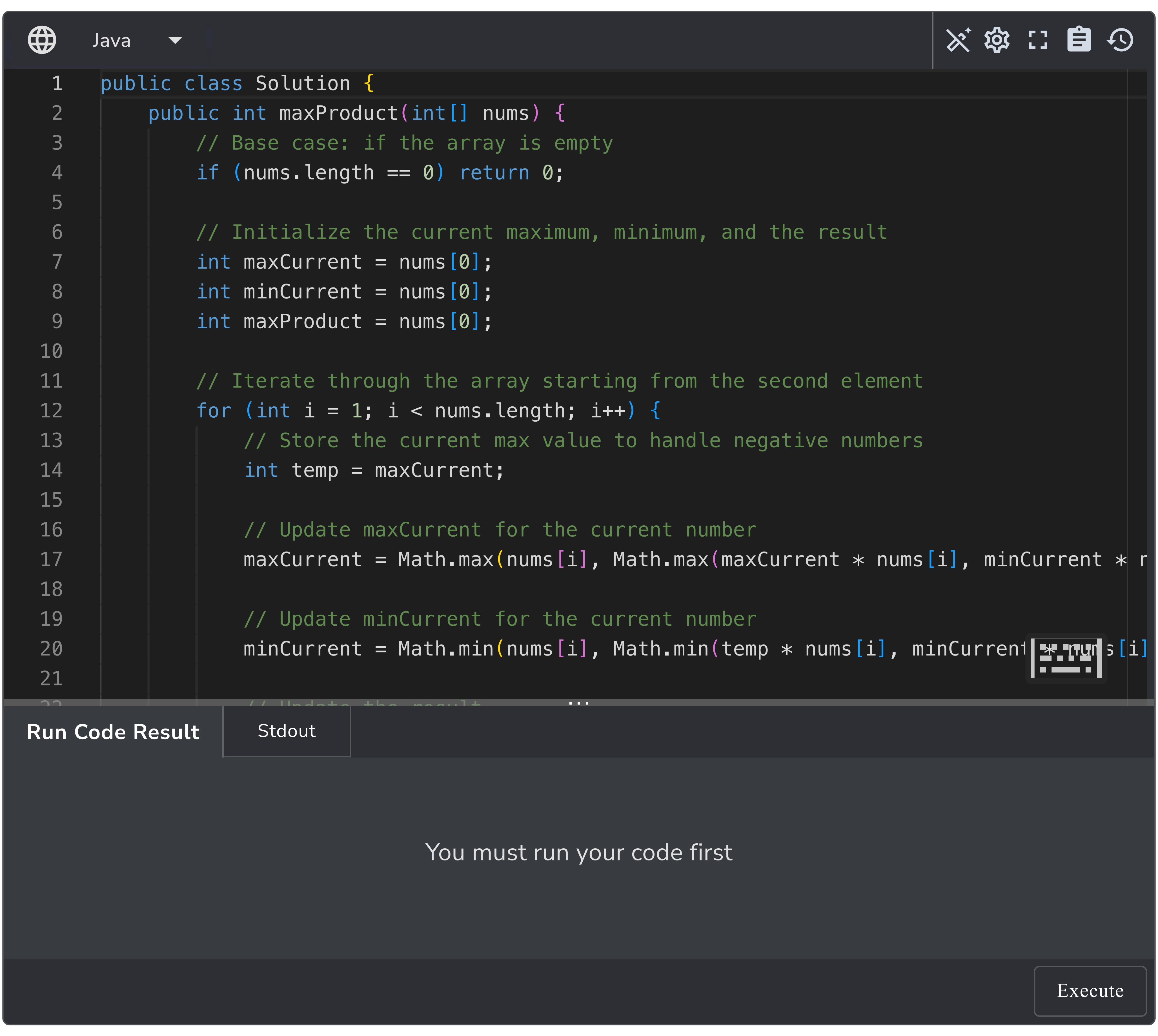
- **Initialization**:
  - Start by initializing two variables, `maxCurrent` and `minCurrent`, to the first element of the array. These variables will keep track of the current maximum and minimum product, respectively.
  - Also, initialize a variable `maxProduct` to the first element of the array. This will store the maximum product found so far.
- **Iterate through the array**:
  - For each number in the array (starting from the second number), calculate the new `maxCurrent` by taking the maximum of the current number, the product of `maxCurrent` and the current number, and the product of `minCurrent` and the current number.
  - Similarly, calculate the new `minCurrent` by taking the minimum of the current number, the product of `maxCurrent` and the current number, and the product of `minCurrent` and the current number.
  - Update `maxProduct` by taking the maximum of `maxProduct` and `maxCurrent`.
- **Handle negative numbers**:
  - Since a negative number can turn a large negative product into a large positive product, we need to keep track of both the maximum and minimum product at each step.
- **Return the result**:
  - After iterating through the entire array, `maxProduct` will have the maximum product of any subarray.

Algorithm Walkthrough

Using the input [2,3,-2,4]:

- Start with `maxCurrent = 2`, `minCurrent = 2`, and `maxProduct = 2`.
- For the next number, 3:
  - New `maxCurrent` = max(3, 2 * 3) = 6
  - New `minCurrent` = min(3, 2 * 3) = 3
  - Update `maxProduct` = max(2, 6) = 6
- For the next number, -2:
  - New `maxCurrent` = max(-2, 3*(-2)) = -2
  - New `minCurrent` = min(-2, 6*(-2)) = -12
  - `maxProduct` remains 6
- For the last number, 4:
  - New `maxCurrent` = max(4, -2*4) = 4
  - New `minCurrent` = min(4, -12*4) = -48
  - `maxProduct` remains 6
- The final answer is 6.

Code



```java
public class Solution {
    public int maxProduct(int[] nums) {
        // Base case: if the array is empty
        if (nums.length == 0) return 0;

        // Initialize the current maximum, minimum, and the result
        int maxCurrent = nums[0];
        int minCurrent = nums[0];
        int maxProduct = nums[0];

        // Iterate through the array starting from the second element
        for (int i = 1; i < nums.length; i++) {
            // Store the current max value to handle negative numbers
            int temp = maxCurrent;

            // Update maxCurrent for the current number
            maxCurrent = Math.max(nums[i], Math.max(maxCurrent * nums[i], minCurrent * r

            // Update minCurrent for the current number
            minCurrent = Math.min(nums[i], Math.min(temp * nums[i], minCurrent * nums[i]

```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n) - We iterate through the array only once.
- **Space Complexity**: O(1) - We use a constant amount of space regardless of the input size.